# Bigtable - Michaël Bettan

| **Definition**: A managed, scalable NoSQL database service for large analytical and operational workloads | **Availability**: Multi-regional or regional | **SLA**: 99.999% |
|---|---|---|

**Use Cases**:
- PB Large analytical workloads (OLTP)
- High throughput analysis & Low latency writes
- Key-based reads
- Time-series workloads (Financial, IoT, etc.)
- HBase legacy migration
- Real-time serving (ad serving, mobile app recommendations, etc.)
- Batch analytics (ML, Data Mining, etc)

**Billing**:
- Type of instance and number of nodes in your clusters
- Amount of storage that your tables use (GiB)
- Amount of network bandwidth that you use (GiB)

Backup: You are billed only for the backup storage that you use, and a backup can never be larger than the original table.

## Key capabilities

- *Wide Column database (multidimensional array. which may contain many columns)*
- *Managed service (nodes provisioning required)*
- *Horizontal scaling*
- *Hbase API compatibility interoperability*
- *Cluster configuration required (nodes, disk, regions, etc)*
- *Adding a new cluster for High-Availability*
- *Operations are atomic at the row level*

## Storage & Architecture Model

- Cell → each row/column can contain multiple cells
- Rows → single entity, indexed by single row-key
- Row key = unique identifier for a row of data stored (index)
  - Table has only one index, based on a row key
- Columns = grouped into families
- Column families = group of related column in a table
- Tablets = Table is sharded into blocks of contiguous rows
  - Smallest logical structure for grouping/storing data
  - Associated with a specific node.
- SSTable = Tablets are stored in Colossus in this format
  - Managed multiple sorted-string tables
  - Persistent, ordered immutable map from keys to values, where both keys and values are arbitrary byte strings
- Nodes = stores the Metadata (SSTables)
- Colossus's shared log → stores all the writes
- Row keys → define Node and SSTables data location
- Cluster → Each node handles a subset of the requests to the cluster

| | Column family 1 | | Column family 2 | |
|---|---|---|---|---|
| | *Column 1* | *Column 2* | *Column 1* | *Column 2* |
| Row key 1 | | | | |
| Row key 2 | | | | |

## Consideration

- *Designing a schema is different from typical RDMS.*
- *Each table has only one index, the row key (4 KB)*
- *Rows are sorted lexicographically by row key.*
- *All operations are atomic (ACID) at row level.*
- *Both reads and writes should be distributed evenly*
- *Try to keep all info for an entity in a single row*
- *Related entities should be stored in adjacent rows*
- *Try to store 10 MB in a single cell (max 100 MB) and 100 MB in a single row (256 MB)*
- *Limitation of 1,000 tables per instance.*
- *Choose row keys that don't follow predictable order*
- *Can use up to around 100 column families*
- *Column Qualifiers: can create as many as you need in each row, but should avoid splitting data across more column qualifiers than necessary (16 KB)*
- *Tables are sparse. Empty columns don't take up any space. Can create large number of columns, evenif most columns are empty in most rows.*
- *Field promotion (shift a column as part of the row key) and salting (remainder of division of hash of timestamp plus row key) are ways to help design row keys.*

## Best Practices

- *Creating many small tables is a Bigtable anti-pattern*
- *Store datasets with similar schemas in the same table, rather than in separate tables.*
- *Time-series data: use tall/narrow tables (one event per row)*
- *Denormalize: prefer multiple tall and narrow tables*
- *Short and wide tables when ???*
- *Designing row-keys is critical step for performance*
- *Good row keys: distributed load*
  - *From common value to a granular value*
  - *Include a timestamp (but not at the beginning)*
- *Poor row keys = hot-spotting when writing*
  - *Sequential identifiers*
  - *Timestamps*
- *Poor row keys:*
  - *Sequential numeric IDs*
  - *Timestamps (start with a timestamp or by itself)*
  - *Row keys that cause related data to not be grouped together*
  - *Frequently updated identifiers*
  - *Hashed values*
  - *Values expressed as raw bytes (rather than human)*
- *Time series data*
- *No support for joining (no foreign key concept)*
- *Distributed reads and writes across nodes*

- *Avoid hot-spotting (on the same node)*
    - *Auto-incrementing for row keys*
    - *Time-value for row keys*
- *Related entities should be stored in adjacent rows to improve read efficiency*
    - *Design row key based on access patterns*
- *Columns are grouped together when frequently access (limitation: up to 100 columns)*
- *Small table are not efficient → wider table*
- *Query patterns*

## Instance configuration

- **Instance** is container for your clusters
- **SSD**: lower latency and more rows read per second. Typically used for real-time serving (e.g., Ad serving, Mobile App reco)
- **HDD:** higher latency for random reads. Good performance on scans and used for batch analytics (e.g., data mining, ML)
- **Changing disk type** requires a **new instance** (HDD <-> SSD)
- **Manual scaling -** Add/Delete nodes and cluster - **without downtime**, but **Autoscaling** was recently introduced.
- **Replication:** add clusters to this instance to automatically replicate data between them (cross-regional supported)
    - Use cases: isolate serving app from batch reads, improve availability, backup, global presence
- **App profile** defines the **routing policy** that Bigtable uses and controls whether **single-row transactions** are allowed.
    - Single-cluster routing → 1 cluster
    - Multi-cluster routing → nearest cluster

## Security Access Control

- *Access level: Project or Instance*
- *Read*
- *Write*
- *Manage*

## Designing Row Keys

- *Design your row keys based on the queries you will use to retrieve the data*
- *Keep your row keys **short** (4 KB or less)*
- *Store multiple delimited values in each row key*
- *Use human-readable string values in your row keys*
- *Design row keys that start with a common value and end with a granular value.*
- *Avoid: Row keys that start with a timestamp*
- *Avoid: Row keys that cause related data to not be grouped.*
- *Avoid: Sequential numeric IDs*
- *Avoid: Hashed values.*

## Miscellaneous

- Related columns should be placed in a column family.
- A single table can have multiple column families.
- Related data should be in one table, not multiple tables.
- Bigtable does not support secondary indexes.
- Row keys are specified for each row, not for each column family.
- Command line - **cbt tool (preferred)** or HBase shell

## Garbage collection

- **Asynchronous and Delayed:** runs continuously in the background but isn't immediate. Data marked for deletion (e.g., by a time-to-live setting) can **take up to a week to be fully removed** from the system.
- **Column Family Specific:** Garbage collection rules are defined at the column family level. Each column family can have its own independent garbage collection policy.
- **Multiple Deletion Criteria:** allows for flexible garbage collection rules. You can configure rules based on the age of data (time-to-live), the number of versions of a cell, or a combination of these criteria.
- **Tombstoning:** When all cells within a row or a column qualifier are deleted, the row or column qualifier is marked as "tombstoned" and eventually removed during garbage collection.
- **Filtering for Accuracy:** Due to the asynchronous nature of garbage collection, queries might return data already marked for deletion. Applying timestamp range filters within queries ensures that only the desired recent data is retrieved, bypassing the garbage collection delay.